```
> function articleInfo()
  {
        this.name="Object Oriented JavaScript";
        this.authorName="Ankur Aggarwal";
  }

  var osfy=new articleInfo();
  undefined
> osfy.name;
  "Object Oriented JavaScript"
> osfy.authorName;
  "Ankur Aggarwal"
> osfy.__proto__;
  ▼articleInfo
    ▼constructor: function articleInfo() {
        arguments: null
        caller: null
        length: 0
        name: "articleInfo"
      ▶prototype: articleInfo
      ▶__proto__: function () {
    ▶__proto__: Object
```

Figure 1: An example of inheritance

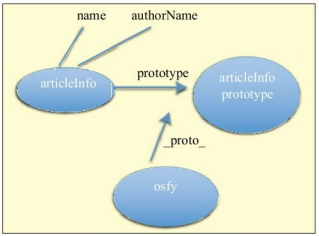

Figure 2: Graphical representation

keyword (inheritance). While you are creating the object using a new keyword, the function will behave as a constructor.

Inheritance

__proto__: Every object in JavaScript will have a property called *__proto__*, which will always point to the parent object. So in the above example, object *obj.__proto__* is pointing to the *info.constructor.*

Prototype: Every function in JavaScript will contain some constructor properties. Further constructor properties will also have prototype properties. This prototype property is the basis of inheritance, and hence the inheritance in JavaScript is referred to as prototypal inheritance.

Consider this figure as an example.

In the code shown in figure 1, *osfy* is the object in which *__proto__* of *osfy* is pointing to the prototype of *articleInfo*. All the constructor properties as well as prototype properties of the *articleInfo* are now inherited by the *osfy*. Check out the last line, *osfy.__proto__*. It clearly shows that it has got all the properties of *articleInfo* and is pointing to the same object, including its constructor and prototype properties.

A graphical representation of the above example is shown in Figure 2.

Prototype chaining

Prototype chaining explains how the inheritance works in JavaScript. Consider Figure 2 for an explanation. Let's suppose you are looking for the name property on the *osfy* object. It will first look in its own properties. If the browser engine is not able to find it in the *osfy* object, then it will move to its *__proto__*, which is pointing to the *articleInfo* object. Then it will look into its constructor properties and there it will find the required name property. If the engine is unable to find the property in the constructor properties also, it will look into prototype properties. So it will keep on looking until it finds null.

An example

```
function person()
{
      this.firstName="Ankur",
      this.lastName="Aggarwal"

}

person.prototype.firstName="Anil";
var p=new person();
p. firstName //prints "ankur"
delete p.firstName;
p. firstName //Anil
```

From the above example it is clear that the flow is:

Object Properties->Parent Object->Constructor Properties->Prototype Properties->.....->null

These are the basics of JavaScript's object-oriented nature, which might have come as a surprise to you. According to me, JavaScript is the most misunderstood language in the computing world. A few months ago, I was using JavaScript just for DOM manipulation but after getting an in-depth knowledge of it, I am in love with it. I can say that JavaScript is a real programming language. So just give it a try and you will come to know about the good parts of JavaScript. Queries and suggestions are always welcome. END

By: Ankur Aggarwal

The author can be contacted at *ankur.aggarwal2390@gmail.com* or followed on Twitter at *www.twitter.com/ankurtwi*. He blogs at *www.flossstuff.wordpress.com*